1. perfindex null run: run report

Run: `null-136x5`, started 2026-10-24T02:08Z (UTC). **Tool:** `examples/perfindex` at repo revision `da3a1b5d5` (tool source hash `fcf8c508…`), PR aburan28/crypto #2478. **Records:** `research/perfindex_validation_20261024/null-136x5/` (`plan.json`, `raw.jsonl`, `compare.json`, `cost.json`, `SHA256SUMS`). This is a descriptive run report: observations only, no claim about any code change.

1.1. Requested scope and what ran

Question. How often does the perfindex verdict rule call a kernel faster or slower when the two arms are the same program? Every such verdict is false by construction.

Command. `perfindex null --bin clean-perfbench --rounds 5 --threads 1 --taskset 2 --samples 5 --max-seconds 1 --weights docs/perf/weights.json` (seed 20260926, schedule hash `a74b35928acc5791`). The candidate arm is a copy of the baseline binary with one trailing byte appended, so the hashes differ (baseline `e0ffb2eb…`, copy `f518c689…`).

Executed (measured). 136 quick-tier kernels × 5 rounds × 3 arms (A, B = copy of A, A2 = a second A) = 2040 invocations, all present exactly once; 1118.2 s inside invocations, 588.6 s of that in timed samples, peak RSS 112,956 kB. The revision the baseline binary was built from was not recorded (`plan.json` has no revision for it).

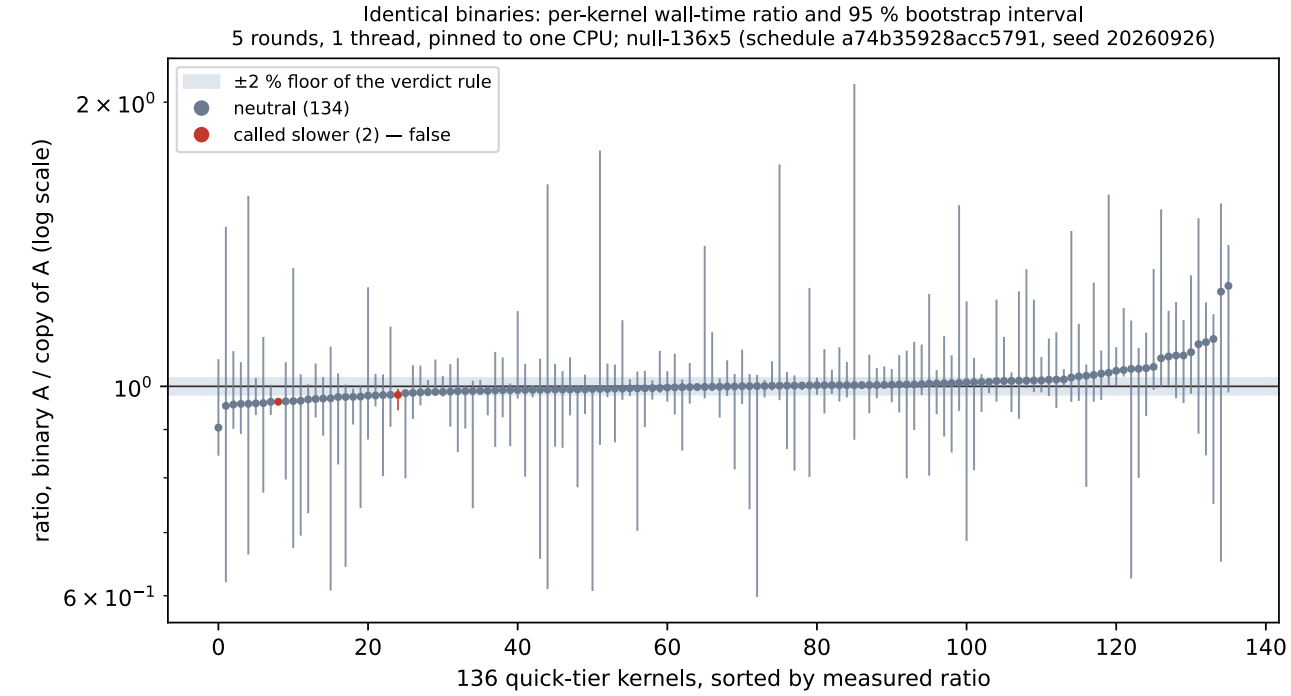
Host (recorded). Intel Xeon 2.80 GHz virtual machine, 4 logical cores, SMT reported off, 16 GB, Linux 6.18.44, load average 1.07 / 1.18 / 1.26 at start; frequency governor not readable. Pinned to CPU 2 with `taskset`; no reserved or isolated core was used. **Isolation level earned: a shared-tenancy virtual machine, pinned, not isolated.** The numbers describe this host under this load only.

1.2. How the recorded summary was produced

The run was first summarised with an involuntary-context-switch limit of 100 per second (`max_invol_per_s: 100`), which flagged 441 of 490 invocations of kernels with a private thread pool pinned to one CPU, and the original summary reported PI 1.006 (95% CI 0.977–1.024), median A/A noise 4.3%, and 61 kernels with contended rounds. That rule was then made opt-in. The committed record is the **same raw rows** re-summarised with the limit off by `perfindex reanalyze` (see `plan.json` field `reanalysis`, which holds the hashes of the original plan, raw rows and summary, and both threshold sets). The committed `compare.json` replays (`perfindex replay`: MATCH). The original summary does **not** replay exactly (its `performance_index_ci95` differs from a recomputation) and is not committed; its hash is in `plan.json`. Both summaries are derived from identical raw rows.

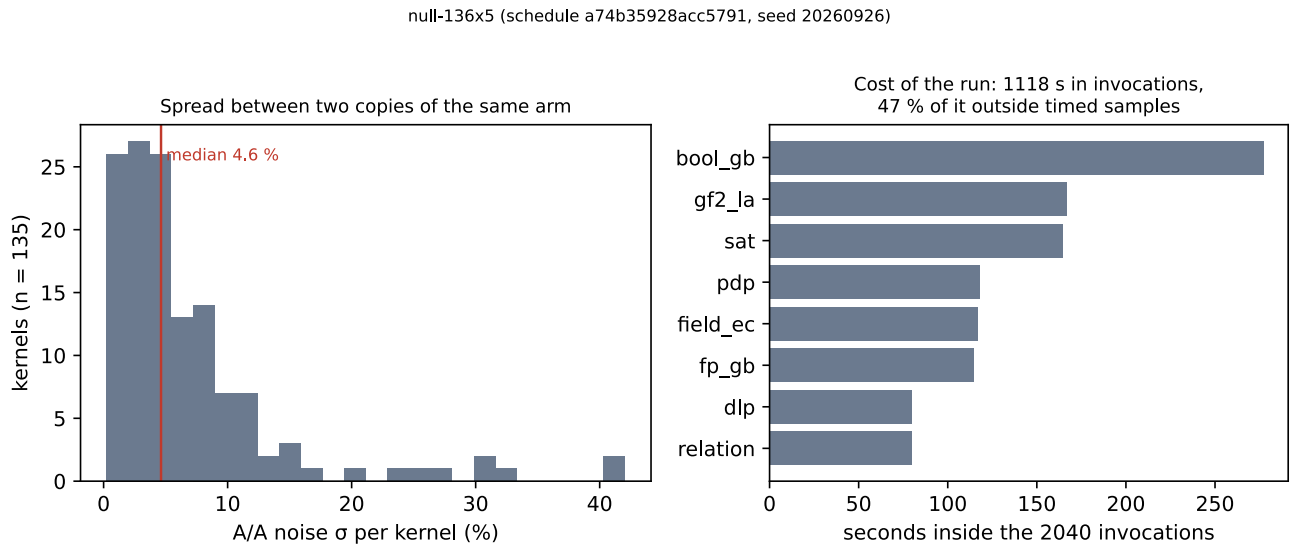
1.3. Observations (all measured or derived from the rows; formulas in `docs/perf/PERFORMANCE_INDEX.md`)

Quantity	Value
Verdicts, 136 kernels	134 neutral, 2 slower, 0 faster (2/136 = 1.5% false verdicts)
Kernels called slower	`fp_gb/f4_solve_quart_n3_p31`: ratio 0.979, 95% CI 0.943–0.994, 4 usable rounds, sign-test p = 0.125, best-of ratio 0.992, **not corroborated** (†). `pdp/descend_s4_n23_np6`: ratio 0.963, CI 0.959–0.970, 4 usable rounds, p = 0.125, best-of 0.960, corroborated.
Performance index (derived)	1.005 (95% bootstrap CI 0.985–1.019), 5 paired rounds; `qualified: false`
Area indices	bool_gb 0.998, dlp 1.020, field_ec 0.999, fp_gb 1.000, gf2_la 1.018, pdp 1.008, relation 0.993, sat 1.005 (each weight 0.125)
A/A noise σ per kernel	median 4.6% over 135 kernels (one kernel has no A/A spread)
Per-kernel ratio, copy vs original	median 0.9995; minimum 0.904 (`relation/pq_sparse_solve_m127_c640`); maximum 1.278 (`dlp/pohlig_hellman_smooth47`); 47 of 136 differ from 1 by more than 2%
95% bootstrap interval excludes 1	9 of 136 kernels (6.6%). Two of the nine (the two "slower" kernels above) also met the rule's magnitude threshold, a median beyond max(2σ, 2%); the other seven did not
Usable rounds	108 kernels 5/5, 28 kernels 4/5; 38 kernels had a contended round left out (hypervisor steal in most cases; CPU time below 90% of wall time in the rest)
Warnings recorded	one verdict not corroborated by best-of; 38 kernels had contended rounds; 1 kernel without A/A spread



Source: compare.json (kernels[].speedup, speedup_ci95, verdict).

Figure 1: Run `null-136x5`: per-kernel ratio of the two identical arms with the 95% bootstrap interval, sorted, 136 kernels, 5 rounds, one thread, pinned to CPU 2. Measured values from `compare.json`. The shaded band is the 2% floor of the verdict rule. Two kernels (red) received a "slower" verdict.



Source: compare.json (aa_noise_ln, aa_noise_median_pct), cost.json (by_area_seconds, invocation_seconds, overhead_fraction).

Figure 2: Run `null-136x5`. Left: A/A noise σ per kernel (135 kernels with an A/A spread). Right: seconds inside the 2040 invocations by area, from `cost.json`; 47% of invocation time was start-up, setup and warm-up rather than timed samples.

1.4. Requirement status

Requirement	Status	Evidence or gap
Run `null` over all quick-tier kernels, 5 rounds, one thread	complete	2040 of 2040 invocations in `raw.jsonl`; `replay` MATCH
Record the false-verdict rate	complete for this host and load	2 of 136; one host, one load, one run
Repeat across hosts, thread counts, loads, or more rounds	not attempted	A single run gives one sample of a rate; no interval on the rate is claimed
Qualified index	not met	`qualified: false`: 28 kernels have only 4 usable rounds and one has no A/A spread
Baseline revision recorded	missing	not in `plan.json` for this run

Full-tier kernels	not run	quick tier only

1.5. Limits

- One run on one shared virtual machine. A false-verdict rate of 2/136 here says nothing about other hosts, loads, round counts, or kernels outside the quick tier.
- Noisy neighbours that cost cache or memory bandwidth without taking CPU time, and frequency changes, are not detectable by the contention rules.
- The two false verdicts both rest on 4 usable rounds, where the smallest possible two-sided sign-test p is 0.125; that is the recorded value, not evidence of an effect.
- The involuntary-switch rule was changed after seeing this run's data. The committed summary reflects the changed default; the original is described above and was not committed.
- Nothing here measures the speed of any cryptanalytic algorithm, so there is no matched rho reference and no row in `benchmarks/README.md`.

1.6. Reproduce

```
cd research/perfindex_validation_20261024/null-136x5
sha256sum -c SHA256SUMS
cargo run --release --example perfindex -- replay .
python3 ../reports/run-20261024T0208Z/plot_null.py   # redraws the figures
```